Project Documentation

1. Overview

Problem Name	Missing Integer
Problem Context	Given an array A of N integers, find the smallest positive integer (greater than 0) that does not occur in A. • Example 1: A = [1, 3, 6, 4, 1, 2] ➔ Returns 5 • Example 2: A = [1, 2, 3] ➔ Returns 4 • Example 3: A = [-1, -3] ➔ Returns 1
Constraints	The solution is designed to meet strict performance requirements: • **Input Size**: Up to **100,000** integers. • **Value Range**: Integers between **-1,000,000** and **1,000,000**.
Project Members	**Yousef.A, Yousef.M, Razan, Rahma, Dina, Lojain**

Objectives	• **Compare Approaches**: Develop and evaluate two different solutions: a **Recursive** method and an **Iterative (Cyclic Sort)** method. • **Optimize Performance**: Ensure the code runs in **Linear Time** to handle large datasets of up to **100,000** elements efficiently. • **Manage Resources**: Minimize memory usage. • **Handle Edge Cases**: Guarantee the algorithm works correctly for negative numbers, zeros, and perfectly sorted sequences.

1. Team Roles

Task	*Assignee*
Write the actual code for the non-recursive (iterative) approach.	**@Razan Abdelkhaleq**
Write the step-by-step Pseudo code for Member 1's algorithm, and Perform Complexity Analysis section.	**@Rahma Essam**

Write the actual code for the recursive approach.	@Yousef El-Ansari
Write the step-by-step Pseudo code for Member 3's algorithm, and Perform Complexity Analysis section.	@Yousef Michael
prepare the comparison between the two algorithms in terms of performance, complexity, and test cases, and write the final document combining all project work.	@Dina Ahmed @Lojain Hussien

1. Pseudocode

3.1 Recursive:

recursive pseudocode

```
algorithm find_miss( cur, pres, max_val )
{
  if (cur > max_val OR NOT pres[cur]) then
        return cur
  return find_miss(cur + 1, pres, max_val)
}

algorithm solve()
{
    write "Enter the number of elements (N): "
    read n
    if reading fails then return
    if n < 0 then n ← 0
    pres ← array of size n +2 filled with FALSE
    write "Enter the elements: "
    for i ← 0 to n - 1 do
    {
        read x
        if (x > 0 AND x ≤ n) then
            pres[x] ← TRUE
    }
    result ← find_miss(1, pres, n + 1)
    write "The smallest missing positive integer is: " , result
}

Main ()
{
    solve()
}
```

3.2 Non-Recursive:

iterative pseudocode

```
1  algorithm MissPos( A, n )
2  // A: input array, n: size of the array
3  {
4     for i ← 0 to n-1 do
5     {
6        while A[i] > 0 AND A[i] ≤ n AND A[i] ≠ A[A[i]-1] do
7        {
8           temp ← A[i]
9           A[i] ← A[temp-1]
10          A[temp-1] ← temp
11       }
12    }
13
14    for j ← 0 to n-1 do
15    {
16       if A[j] ≠ j+1 then
17          return j+1
18    }
19
20    return n + 1
21 }
```

1. Code

4.1 Recursive:

recursiveMissingInt.cpp

```cpp
#include <bits/stdc++.h> //  المكتبة اللي فيها كل العدة
using namespace std;

#define ll long long
#define forn(i, n) for (int i = 0; i < int(n); i++)//  اختصار عشان متعددش تكتب كتير وتذهب صوابحا
#define el "\n"

//  فانكشن الريكرجن دي بقى اللي بتفصل تدور ورا الرقم لحد ما تجيبه من هناه
//  عشان الميموري غالية علينا ومش عايزين نميزقها وناخد منها نسخ كتير  استخدمنا الـ (&)
int find_miss(int cur, vector<bool> &pres, int max_val) {

    //  لو وصلنا للآخر خالص أو لقينا مكان "فاضي" في الخريطة يبقى هو ده الرقم اللي إحنا بـ معلم
    if (cur > max_val || !pres[cur]) {
        return cur;
    }

    //  "لو لسه ملقيناش، بنادي الفانكشن تاني ونقولها: "شوفي اللي بعده يا ست الكل
    return find_miss(cur + 1, pres, max_val);
}

void solve() {
    int n;

    //  بنطلب من اليوزر يدخل عدد العناصر بكل أدب وذباكة
    cout << "Enter the number of elements (N): " ;
    if (!(cin >> n)) return;

    //  لو اليوزر حب يهزر ودخل رقم سالب، هنخيره صفر ونمشي حالنا
    if (n < 0) n = 0;

    //  بنجهز bool كلها كتب لحد ما نلاقي الرقم ونخليها صح
    vector<bool> pres(n + 2, false);

    if (n > 0) {
        cout << "Enter the " << n << " element/s separated by space: " ;
        forn(i, n) {
            int x;
            cin >> x;

            //  بنفلتر الأرقام فا الموجب بس واللي "ع الحركروك" جوه نطاق الأراي هو اللي يهمنا
            if (x > 0 && x <= n) {
                pres[x] = true;//  علم عليه يا باشا إنه جه وشرفنا
            }
        }
    }

    //  اللحظة الحاسمة.. بنطلع النتيجة النهائية لليوزر
    cout << "The smallest missing positive integer is: "  << find_miss(1, pres, n + 1) << el;
}

int main() {
    //  بنفصل الربط مع C الزربو بتاع الكود.. عشان المدخلات والمخرجات تجري طيارة
    //  بنفصل cin عن cout عشان كل واحد يشتغل في سكة لوحده بدون ما يستنى التاني
    //ios_base::sync_with_stdio(false);
    //cin.tie(NULL);

    solve();//  توكلنا على الله

    return 0;//  تم بحمد الله والبرنامج زي الفل
}
```

4.2 Non-Recursive:

iterativeMissingInt.cpp

```cpp
#include <bits/stdc++.h> ///  النسخة اللي فيها كل العدة والآلات الحادة
using namespace std;

///   الفانكشن دي بقى اللي هتخلص المصلحة "من غير ريكرجن" ومن غير وجع دماغ
///   ضفنا الـ & هنا عشان نعدل في الاراي الأصلية ونوفر ميموري (O(1) Space)
int miss_pos(vector<int> &A) {
    int n = A.size();


  ///   أول لفة: بنفرز الأرقام وكل واحد يروح بقعد في بيتهم
    for (int i = 0; i < n; i++) {
      ///   طول ما الرقم موجب و على الحركروك جوه المصفوفة ومش في مكانه.. بدله يا معلم
        while (A[i] > 0 && A[i] <= n && A[i] != A[A[i] - 1]) {
            int temp = A[i];       ///   بنركن الرقم ده على جنب ثانية
            A[i] = A[temp - 1];    ///   بنجيب اللي في مكانه الصح هنا
            A[temp - 1] = temp;    ///   ونبعته هو لبيته اللي بستاهله
        }
    }

  ///   ثاني لفة: بنفتش المصفوفة بيت بيت ونشوف مين اللي لسه "غايب"
    for (int i = 0; i < n; i++) {
        if (A[i] != i + 1) {
            return i + 1; ///   لقينا الرقم الغايب.. يرجع فوراً
        }
    }

  ///   لو كله تمام، يبقى الرقم اللي عليه الدور هو اللي ناقص
    return n + 1;
}

int main() {
    //  شيلنا الزربو عشان التكست يظهر للبوزر في وقته الصح
    //ios_base::sync_with_stdio(false);
    //cin.tie(NULL);
    int n;
    cout << "Enter the number of elements (N): " ;
    if (!(cin >> n)) return 0;

    if (n <= 0) {
        cout << "The smallest missing positive integer is: 1"   << endl;
        return 0;
    }

    vector<int> A(n);
    cout << "Enter the " << n << " element/s separated by space: " ;
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    cout << "The smallest missing positive integer is: " ;
    cout << miss_pos(A) << endl;

    return 0;  ///  تم بحمد الله وكله زي الفل
}

```

1. Testing

5.1 Recursive:

Testing Logic	*Input*	*Expected Output*	*Output*
boundary n+1	**[1 ,2 ,3]**	**4**	**4**
General gap detection	**[1, 3, 6, 4, 1, 2]**	**5**	**5**
Ignoring non-positives	**[-1 ,-3]**	**1**	**1**
Handling repetition	**[1 , 1 , 2 , 4]**	**3**	**3**

Single Element (Best Case for search)	[1]	2	2
Empty Input	[ ]	1	1
Zero exclusion	[0 , 2 , 3]	1	1
Out-of-bounds values	[1 , 500 , 2]	3	3

5.2 Non-Recursive:

Testing Logic	*Input*	*Expected Output*	*Output*
boundary n+1	**[1 ,2 ,3]**	**4**	**4**
General gap detection	**[1, 3, 6, 4, 1, 2]**	**5**	**5**
Ignoring non-positives	**[-1 ,-3]**	**1**	**1**
Handling repetition	**[1 , 1 , 2 , 4]**	**3**	**3**
Single Element (Best Case for search)	**[1]**	**2**	**2**
Empty Input	**[ ]**	**1**	**1**

Zero exclusion	[0 , 2 , 3]	1	1
Out-of-bounds values	[1 , 500 , 2]	3	3

5.3 Authentic Screenshots

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 1 2 3
The smallest missing positive integer is: 4
```

```
Enter the number of elements (N): 6
Enter the 6 element/s separated by space: 1 3 6 4 1 2
The smallest missing positive integer is: 5
```

```
Enter the number of elements (N): 2
Enter the 2 element/s separated by space: -1 -3
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 4
Enter the 4 element/s separated by space: 1 1 2 4
The smallest missing positive integer is: 3
```

```
Enter the number of elements (N): 1
Enter the 1 element/s separated by space: 1
The smallest missing positive integer is: 2
```

```
Enter the number of elements (N): 0
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 0 2 3
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 1 500 2
The smallest missing positive integer is: 3
```

1. Complexity

6.1 Recursive:

6.1.1 Time : O(n)

- Solve 1 (Logic & Basic Operations):

We calculate time complexity by counting how many basic operations the computer performs based on the input size **N**.

Step 1 (Initializing the array):

```
vector<bool> pres(n + 2, false);
```

To create a vector of size N+2 and set all values to false, the computer visits every single slot.

Time taken: O(N)

Step 2 (Reading and filtering the input):

The `forn(i, n)` loop runs exactly N times. Inside the loop , checking `x > 0 && x <= n` and assigning `pres[x] = true` are instant , basic math operations. Since it does a basic operation N times, the time scales linearly with the input.

Time taken: O(N)

Step 3 (The Recursive Search):

```
find_miss(1, pres, n + 1)
```

In the **Worst-Case Scenario** (for example, if your array is A = [1, 2, 3, 4, 5]), the function makes exactly N+1 recursive calls to find the missing integer.

Time taken: O(N)

Total Time: O(N) + O(N) + O(N) = O(3N), which is **O(n)**.

- **Solve 2 (using iteration method):**

We can represent the recursion with the following relation:

- T(n) = T(n-1) + c
- Substitute T(n-1) : T(n) = (T(n-2) + c) + c = T(n-2) + 2c
- Substitute T(n-2) : T(n) = (T(n-3) + c) + 2c = T(n-3) + 3c
- If we continue this pattern for k steps, we get the general form: **T(n) = T(n-k) + kc**
- The base case is reached when n - k = 0, meaning k = n.
- Substituting n for k: **T(n) = T(n-n) + nc = T(0) + nc**
- Since T(0) takes a constant amount of time, we drop the constants, concluding that T(n) = Θ(n).

6.1.2 Space : O(N)

Note from Lecture: Running time is affected by a number of factors other than algorithm efficiency

So, Instead of timing an algorithm, count the number of instructions that it performs

- **Step 1 (The Boolean Array):**

```
vector<bool> pres(n + 2, false);
```

Creating a brand new list in memory directly tied to input size (N+2).

Space used: $O(N)$

- Step 2 (The Recursion Stack):

```
return find_miss(cur + 1, pres, max_val);
```

Every time a function calls itself recursively, the computer has to pause the current function, save its place in memory, and open a new function. This creates a "**Call Stack**."

In the worst-case scenario (like the 1, 2, 3, 4, 5 example), the function calls itself N + 1 times before hitting the base case. The computer has to keep all N + 1 function calls open in memory at the exact same time.

Space used: $O(N^2)$

Total Space Complexity: $O(N^2)$

$O(N)$ (Array) + $O(N^2)$ (Stack) → **Final Space is** $O(N^2)$.

6.2 Non-Recursive:

6.2.1 Time : O(n)

Step 1 (Sorting): We have a for loop from 0 to n-1 with a while loop performs a manual swap

to put each number in its correct place. Even though it's a loop inside a loop, each number is

moved to its correct position only once. Therefore, the total number of operations is

proportional to n. So, this step takes O(n).

Step 2 (Scanning): We use a simple for loop to check the numbers one by one, which also

takes O(n).

Total Time: O(n)+O(n)=O(2n), which is **O(n)**.

6.2.2 Space : O(1)

The algorithm is In-place. We rearrange the numbers inside the same array A.

We do not use any extra arrays or HashSets.

We only use a few simple variables like n, i and temp, so the extra space is constant.

Total Space: O(1) .

1. Comparison

Criteria	Non-Recursive	Recursive
Time complexity	O(n)	O(n)
Space complexity	O(1)	$O(n^2)$
Memory Usage	Very Low (Uses original array)	High (Uses extra vector & stack)
Implementation	Needs a bit of logic (Swapping)	Easier to think of (Intuitive)
Scalability	Highly Scalable	Limited by Stack Size (May Crash)
Approach	**In-place Sorting**: Uses the array itself as a map by moving each number to its	**Indirect Marking**: Uses an external boolean map to track existing numbers.

	index.	
Readability	***Moderate**: Requires understanding of the Cyclic Sort swap logic (more "Engine-like").*	***High**: The code is clean, intuitive, and follows a "Human-like" logic.*
Risks	***Data Mutation**: It modifies the original input array (might be a risk if data is needed later).*	***Stack Overflow**: Risk of crashing if N is very large (e.g., 10^6) due to stack depth.*

7a) Performance Insight:

The performance of an algorithm is not just about how fast it runs, but how it manages system resources (Time & Memory).

- **Time Efficiency:** Both algorithms are efficient with a Linear Time Complexity O(n). This means they can handle large datasets without a massive drop in speed.
- **Memory Efficiency:** This is where they differ significantly. The Recursive approach is "Memory-Expensive" because it requires extra space for the boolean array and the call stack. On the other hand,

the Non-Recursive (Cyclic Sort) is "Memory-Efficient" as it works In-place, using only the existing array.

- **Safety:** The Non-Recursive approach is more stable for "Big Data" because it avoids the risk of a Stack Overflow, which is a common failure point for deep recursion.

7b) When to use each?

Choosing the right algorithm depends on the specific requirements of your project:

Use the Recursive Approach when:

- **Simplicity is Priority:** When the code needs to be written quickly and must be very easy for other developers to read and maintain.
- **Small Data:** When you know the input size N is small, so the memory usage of the stack won't be an issue.
- **Prototyping:** Ideal for academic assignments or quick proofs of concept where clean logic is more important than hardware optimization.

Use the Non-Recursive (Cyclic Sort) when:

- **Memory is Limited:** Essential for embedded systems, mobile apps, or any environment with restricted RAM.
- **Massive Datasets:** When dealing with millions of elements where recursion would definitely crash the system.
- **Performance-Critical Systems:** When you need the **Optimal** solution (O(n) Time and O(1) Space) to ensure the highest possible efficiency.

1. Another Alternatives

8.1 Rejected Alternatives for the Iterative Solution:

8.1.1 Standard Sorting (O(N log N) Time, O(1) Space)

Approach: Sort the array using std::sort, then linearly scan to find the first gap.

Reason for Rejection: The problem explicitly requires an efficient $O(N)$ time complexity algorithm. Sorting takes $O(N \log N)$, which is mathematically slower and sub-optimal for this specific problem.

8.1.2 Hash Set / External Boolean Array (O(N) Time, O(N) Space)

Approach: Use a secondary data structure to mark the presence of elements.

Reason for Rejection: It violates the optimization goal. The assignment implicitly favors an auxiliary space approach (like Cyclic Sort) to achieve O(1) constant space. Using a Hash Set consumes unnecessary memory resources.

8.1.3 Brute Force (O(N^2) Time, O(1) Space)

Approach: For every integer starting from 1, scan the entire array to check if it exists.

Reason for Rejection: Extremely inefficient. For an input of 10^5 elements, this would result in 10^10 operations, inevitably causing a **Time Limit Exceeded (TLE)** error.

8.2 Rejected Alternatives for the Recursive Solution:

8.2.1 Divide and Conquer / Partitioning (O(N) Time)

Approach: Use a Quick-Sort-like partitioning to move negatives and out-of-bound numbers, then recursively search the valid half.

Reason for Rejection: Over-engineering. This approach is notoriously complex to implement for this specific problem. Our Linear Recursion with a frequency array achieves the same O(N) goal with significantly cleaner and more readable code.

8.2.2 Recursion with Pass-by-Value (Missing the &)

Approach: Passing the vector to the recursive function without the reference operator (&).

Reason for Rejection: Critical Memory Overhead. Passing by value creates a full copy of the array for every recursive call. If the recursion depth is N, this results in O(N^2) time and O(N^2) space, guaranteeing a **Memory Limit Exceeded (MLE)** or **Stack Overflow** crash.

8.2.3 In-place Recursive Swapping

Approach: Implementing the Cyclic Sort swapping logic recursively.

Reason for Rejection: High risk of infinite loops (especially with duplicate values) and extremely difficult to debug. Separating the recursion to just "Read-Only" logic is much safer and maintains code purity.